B.Bhanu Prasad

Batch no:38

2303a52021

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
   - o Identify the logical bug in the code.
   - o Explain why the bug occurs (e.g., off-by-one error).
   - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

AIAC_LAB 10.3.py > ...

```python
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result = result * i
    return result

# Test the function
print(factorial(5))

# The bug was in the range(1, n) which should be range(1, n+1)
# range(1, n) only goes up to n-1, missing the final multiplication by n
# For factorial(5): should multiply 1*2*3*4*5 = 120

# Edge case handling:
# Consider adding validation for negative numbers and zero:
def factorial_safe(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers")
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(1, n + 1):
        result = result * i
    return result

# Test cases
print(factorial_safe(5))   # 120
print(factorial_safe(0))   # 1
print(factorial_safe(1))   # 1
```

OUTPUT:

Debug Console (Ctrl+Shift+Y)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Bhanu Prasad\AIAC LAB> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 10.3.py"
120
120
1
1
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

Explanation:

In the above we are calculating the factorial of a number with function.

The function returns the factorial of number .

Problem Statement 2: Task 2 — Improving Readability &

Documentation

Scenario:The following code works but is poorly written:

.

```
def calc(a, b, c):

if c == "add":

return a + b

elif c == "sub":

return a - b
```

```
elif c == "mul":
return a * b
elif c == "div":
```

Instructions:

5. Use AI to:

o Critique the function’s readability, parameter naming, and lack of documentation.

o Rewrite the function with:

1. Descriptive function and parameter names.
2. A complete docstring (description, parameters, return value, examples).
3. Exception handling for division by zero.
4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles errors gracefully.

```python
def perform_arithmetic_operation(operand1, operand2, operation):
    Performs basic arithmetic operations on two numbers.

    Parameters:
        operand1 (float): The first number for the operation.
        operand2 (float): The second number for the operation.
        operation (str): The operation to perform ('add', 'sub', 'mul', 'div').

    Returns:
        float: The result of the arithmetic operation.

    Raises:
        ValueError: If operation is not supported or operand types are invalid.
        ZeroDivisionError: If attempting to divide by zero.

    Examples:
        >>> perform_arithmetic_operation(10, 5, 'add')
        15
        >>> perform_arithmetic_operation(10, 5, 'div')
        2.0
    """
    if not isinstance(operand1, (int, float)) or not isinstance(operand2, (int, float)):
        raise ValueError("Operands must be numbers")

    if operation == "add":
        return operand1 + operand2
    elif operation == "sub":
        return operand1 - operand2
```

```python
    elif operation == "div":
        if operand2 == 0:
            raise ZeroDivisionError("Cannot divide by zero")
        return operand1 / operand2
    else:
        raise ValueError(f"Unsupported operation: {operation}. Use 'add', 'sub', 'mul', or 'div'.")

# Test cases
print(perform_arithmetic_operation(10, 5, 'add'))    # 15
print(perform_arithmetic_operation(10, 5, 'div'))    # 2.0
try:
    print(perform_arithmetic_operation(10, 0, 'div'))
except ZeroDivisionError as e:
    print(f"Error: {e}")
try:
    print(perform_arithmetic_operation(10, 5, 'pow'))
except ValueError as e:
    print(f"Error: {e}")
```

OUTPUT:

```
PS C:\Users\Bhanu Prasad\AIAC LAB> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 10.3.py"
15
2.0
Error: Cannot divide by zero
Error: Unsupported operation: pow. Use 'add', 'sub', 'mul', or 'div'.
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

Explanation:

In the code we declared different functions to operate basic operations like add,sub,multi,division.And also when the number is divided by zero it is rasing the error to identify that error by user.

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
   - List all PEP8 violations.
   - Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

```python
def check_prime(n):
    """
    Check if a number is prime.

    Parameters:
        n (int): The number to check.

    Returns:
        bool: True if the number is prime, False otherwise.

    Examples:
        >>> check_prime(5)
        True
        >>> check_prime(4)
        False
    """
    if not isinstance(n, int) or n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```python
# Test cases
print(check_prime(2))    # True
print(check_prime(5))    # True
print(check_prime(4))    # False
print(check_prime(1))    # False
print(check_prime(17))   # True
print(check_prime(20))   # False

# Note on AI-assisted Code Reviews:
# Automated AI reviews streamline large team code reviews by:
# 1. Enforcing consistent style standards (PEP8, naming conventions)
# 2. Catching common bugs and logic errors automatically
# 3. Reducing manual review time for style issues
# 4. Providing educational feedback for junior developers
# 5. Enabling reviewers to focus on architectural and logic concerns
# 6. Maintaining code quality consistently across projects
```

OUTPUT:

```
Error: Cannot divide by zero
Error: Unsupported operation: pow. Use 'add', 'sub', 'mul', or 'div'.
PS C:\Users\Bhanu Prasad\AIAC LAB> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 10.3.py"
True
True
False
False
True
False
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:

o Readability and naming.

o Reusability and modularity.

o Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:

a. Better naming and function purpose clarity.

b. Input validation and type hints.

c. Suggestions for generalization (e.g., configurable multiplier).

3. Refactor the function based on AI feedback.

4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent, e.g.:

```
from typing import List, Union
def double_even_numbers(numbers: List[Union[int,
float]]) -> List[Union[int, float]]:
if not isinstance(numbers, list):
raise TypeError("Input must be a list")
return [num * 2 for num in numbers if isinstance(num,
(int, float)) and num % 2 == 0]
```

```python
def apply_operation_to_filtered(
    data: List[Union[int, float]],
    multiplier: Union[int, float] = 2,
    filter_func=None
) -> List[Union[int, float]]:
    """
    Apply a multiplier to filtered elements in a list.

    Parameters:
        data: List of numbers to process.
        multiplier: Value to multiply filtered elements by (default: 2).
        filter_func: Optional function to filter elements (default: even numbers).

    Returns:
        List of multiplied, filtered numbers.

    Raises:
        TypeError: If input is not a list or contains non-numeric values.
        ValueError: If multiplier is not numeric.
    """
    if not isinstance(data, list):
        raise TypeError("Input must be a list")

    if not isinstance(multiplier, (int, float)):
        raise ValueError("Multiplier must be numeric")
```

```python

    if not all(isinstance(x, (int, float)) for x in data):
        raise TypeError("All list elements must be numeric")

    if filter_func is None:
        filter_func = lambda x: x % 2 == 0

    return [x * multiplier for x in data if filter_func(x)]


# Test cases
print(apply_operation_to_filtered([1, 2, 3, 4, 5, 6]))  # [4, 8, 12]
print(apply_operation_to_filtered([1, 2, 3, 4], multiplier=3))  # [6, 12]
print(apply_operation_to_filtered([1, 2, 3, 4], filter_func=lambda x: x > 2))  # [6, 8]
```

OUTPUT:

```
PS C:\Users\Bhanu Prasad\AIAC LAB> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 10.3.py"
[4, 8, 12]
[6, 12]
[6, 8]
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

Explanation:

In the above code it takes the list of number and perform the filter function then perforn the multiplication function

AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but

it runs slowly on large datasets:

```
def sum_of_squares(numbers):

total = 0

for num in numbers:

total += num ** 2

return total
```

Instructions:

1. Test the function with a large list (e.g., range(1000000)).

2. Use AI to:

o Analyze time complexity.

o Suggest performance improvements (e.g., using built-in

functions, vectorization with NumPy if applicable).

o Provide an optimized version.

3. Compare execution time before and after optimization.

4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):

return sum(x * x for x in numbers)
```

```python
import numpy as np
import time

def sum_of_squares(numbers):
    """Original function - O(n) time complexity"""
    total = 0
    for num in numbers:
        total += num ** 2
    return total

def sum_of_squares_optimized(numbers):
    """Optimized with built-in sum - O(n) but faster in practice"""
    return sum(x * x for x in numbers)

def sum_of_squares_numpy(numbers):
    """NumPy vectorized version - fastest for large datasets"""
    return int(np.sum(np.array(numbers) ** 2))

# Test with large dataset
test_data = list(range(1000000))

# Benchmark original
start = time.time()
result1 = sum_of_squares(test_data)
time1 = time.time() - start
print(f"Original: {result1}, Time: {time1:.4f}s")
```

```python
# Benchmark optimized
start = time.time()
result2 = sum_of_squares_optimized(test_data)
time2 = time.time() - start
print(f"Optimized: {result2}, Time: {time2:.4f}s")

# Benchmark NumPy
start = time.time()
result3 = sum_of_squares_numpy(test_data)
time3 = time.time() - start
print(f"NumPy: {result3}, Time: {time3:.4f}s")

print(f"\nSpeedup (optimized vs original): {time1/time2:.2f}x")
print(f"Speedup (NumPy vs original): {time1/time3:.2f}x")
```

OUTPUT:

```
... Original: 333332833333500000, Time: 0.1031s
    Optimized: 333332833333500000, Time: 0.0728s
    NumPy: 333332833333500000, Time: 0.0811s

    Speedup (optimized vs original): 1.42x
    Speedup (NumPy vs original): 1.27x
```

Expalnation:

The code defines and benchmarks three Python functions for calculating the sum of squares of numbers: a basic loop, an optimized version using sum and a vectorized

NumPy version. It then prints the execution time and speedup for each method against a large dataset.